

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I **Challa Priyadarshini(192311118)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ch.Priyadarshini

CRYPTOGRAPHY AND NETWORK SECURITY

Python Implementations, Demonstrations and Security Analysis

1. Implement and Compare MD5 and SHA-256

Aim

To implement MD5 and SHA-256 hashing in Python, calculate digests for multiple messages, compare their output characteristics and execution time, and analyze their collision resistance.

Theory

A cryptographic hash function converts an input of arbitrary length into a fixed-size digest. A good cryptographic hash should provide preimage resistance, second-preimage resistance and collision resistance, and should exhibit the avalanche effect.

Algorithm

1. Read a collection of input messages.
2. Encode each message as UTF-8 bytes.
3. Calculate its MD5 digest using `hashlib.md5()`.
4. Calculate its SHA-256 digest using `hashlib.sha256()`.
5. Display both digests.
6. Hash a large data block and measure execution time with `time.perf_counter()`.
7. Compare security and performance observations.

Python Program

```
import hashlib
import time
messages = [
    "Hello World",
    "Cryptography",
    "Network Security",
    "The quick brown fox jumps over the lazy dog",
    "Python Laboratory"
]
print("=" * 80)
print("MD5 AND SHA-256 COMPARISON")
print("=" * 80)
for message in messages:
    md5_value = hashlib.md5(message.encode("utf-8")).hexdigest()
    sha256_value = hashlib.sha256(message.encode("utf-8")).hexdigest()
    print("\nMessage :", message)
    print("MD5    :", md5_value)
    print("SHA256 :", sha256_value)
# Performance experiment
```

```

data = b"A" * (10 * 1024 * 1024) # 10 MB
start = time.perf_counter()
hashlib.md5(data).digest()
md5_time = time.perf_counter() - start
start = time.perf_counter()
hashlib.sha256(data).digest()
sha256_time = time.perf_counter() - start
print("\n" + "-" * 80)
print("Performance for 10 MB data")
print("MD5 time   :", md5_time, "seconds")
print("SHA-256 time:", sha256_time, "seconds")
if md5_time < sha256_time:
    print("Observation : MD5 was faster in this run.")
else:
    print("Observation : SHA-256 was faster in this run.")
print("Note: Timing depends on CPU, Python version and cryptographic backend.")

```

Sample Output

```

MD5 AND SHA-256 COMPARISON
Message : Hello World
MD5   : b10a8db164e0754105b7a99be72e3fe5
SHA256 : a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b...
...
Performance for 10 MB data
MD5 time   : [system dependent]
SHA-256 time: [system dependent]
Observation : [depends on the machine]

```

The exact performance values vary between computers, so they should be obtained by running the program on the target system.

Security Analysis

Property	MD5	SHA-256	Conclusion
Digest size	128 bits	256 bits	SHA-256 provides a larger digest
Collision resistance	Broken	Strong for current practical use	SHA-256 preferred
Generic collision work	$\approx 2^{64}$	$\approx 2^{128}$	SHA-256 has much higher margin
Speed	Often faster	Often slower	Depends on hardware/backend
Modern security use	Not recommended	Recommended	Use SHA-256 or stronger

Result

MD5 and SHA-256 were successfully implemented and compared. MD5 is faster in many environments, but its collision resistance is cryptographically broken. SHA-256 provides substantially stronger collision resistance and is suitable for modern security applications.

2. RSA Digital Signature Scheme

Aim

To implement an educational RSA digital signature scheme in Python, sign a message with a private key, verify it with the corresponding public key, and demonstrate detection of message tampering.

Theory

In a digital signature system, the signer uses a private key to generate a signature. A verifier uses the public key to check the signature. A secure signature scheme provides message integrity and authentication of the private-key holder, provided that the public key is trusted.

Python Program

```
import hashlib
# Educational RSA parameters only
p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)
e = 17
d = pow(e, -1, phi)
def hash_to_integer(message):
    digest = hashlib.sha256(message.encode("utf-8")).digest()
    return int.from_bytes(digest, "big") % n
def sign(message):
    h = hash_to_integer(message)
    return pow(h, d, n)
def verify(message, signature):
    h = hash_to_integer(message)
    recovered_hash = pow(signature, e, n)
    return recovered_hash == h
message = "This is an original message"
signature = sign(message)
print("RSA Digital Signature")
print("-----")
print("Public Key :", (e, n))
print("Private Key:", (d, n))
print("Message   :", message)
print("Signature  :", signature)
print("Verification:", verify(message, signature))
```

```
tampered = "This is a modified message"
print("\nTampered Message:", tampered)
print("Verification  :", verify(tampered, signature))
```

Expected Output

```
RSA Digital Signature
-----
Public Key : (17, 3233)
Private Key: (2753, 3233)
Message   : This is an original message
Signature : [integer]
Verification: True
Tampered Message: This is a modified message
Verification  : False
```

Analysis

- Integrity: if the message is changed, the recomputed hash differs and verification fails.
- Authentication: a valid signature indicates that the corresponding private key was used, assuming the public key is authentic.
- Public verification: unlike HMAC, the verifier does not need the signer's private key.
- Non-repudiation: signatures can support non-repudiation in appropriate systems, but this depends on key ownership, certificates and operational controls.
- The example uses very small RSA values for learning only. Production RSA requires large secure keys and standardized padding such as RSA-PSS.

Result

The RSA signature successfully verified the original message and rejected the tampered message, demonstrating integrity and authentication.

3. HMAC Using SHA-256 and Comparison with Simple Hash

Aim

To simulate HMAC using SHA-256 and compare its security properties with a simple SHA-256 integrity check.

Theory

A normal hash is not an authenticator because an attacker who changes a message can calculate a new hash. HMAC uses a secret key together with a hash function. Without the secret key, an attacker cannot normally create a valid MAC for a modified message.

Python Program

```
import hashlib
import hmac
message = b"Transfer Rs.500 to Account123"
secret_key = b"my_shared_secret_key"
```

```

# Simple hash
simple_hash = hashlib.sha256(message).hexdigest()

# HMAC-SHA256
original_hmac = hmac.new(
    secret_key, message, hashlib.sha256
).hexdigest()
print("Message      :", message.decode())
print("SHA-256      :", simple_hash)
print("HMAC-SHA-256 :", original_hmac)
def verify_hmac(message, received_mac):
    calculated = hmac.new(
        secret_key, message, hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(calculated, received_mac)
print("\nOriginal verification:",
    verify_hmac(message, original_hmac))
tampered = b"Transfer Rs.5000 to Account123"
print("Tampered verification:",
    verify_hmac(tampered, original_hmac))
# Demonstrate why a plain hash is weaker for authentication
attacker_hash = hashlib.sha256(tampered).hexdigest()
print("\nAttacker can calculate new plain hash:", attacker_hash)

```

Comparison

Feature	SHA-256 only	HMAC-SHA-256
Detects accidental modification	Yes	Yes
Uses secret key	No	Yes
Provides shared-key authentication	No	Yes
Attacker can recalculate after editing	Yes	Not without secret key
Provides public-key verification	No	No
Provides non-repudiation	No	No

Result

HMAC-SHA-256 successfully detected tampering and provided message authentication using a shared secret. A plain SHA-256 hash can provide integrity checking but cannot prove that a particular party generated the message.

4. Simplified Kerberos Authentication Mechanism

Aim

To simulate the basic idea of Kerberos ticket-based authentication, including ticket generation, distribution, validation, expiration and replay detection.

Theory

Kerberos is a trusted third-party authentication protocol. A client authenticates through a Key Distribution Center (KDC) and receives a ticket that can be presented to a service server. Tickets contain authentication information and have limited lifetimes. Real Kerberos also uses authenticators, session keys and synchronized clocks.

Python Program

```
import hashlib
import hmac
import json
import base64
import time
import secrets

SERVER_KEY = b"server-secret-key"
used_tickets = set()

def create_ticket(client_id, lifetime=30):
    ticket_id = secrets.token_hex(16)
    data = {
        "client_id": client_id,
        "ticket_id": ticket_id,
        "issued_at": int(time.time()),
        "expires_at": int(time.time()) + lifetime
    }
    raw = json.dumps(data, sort_keys=True).encode()
    mac = hmac.new(
        SERVER_KEY, raw, hashlib.sha256
    ).hexdigest()
    encoded = base64.urlsafe_b64encode(raw).decode()
    return encoded + "." + mac

def validate_ticket(ticket):
    try:
        encoded, received_mac = ticket.rsplit(".", 1)
        raw = base64.urlsafe_b64decode(encoded)
        expected_mac = hmac.new(
            SERVER_KEY, raw, hashlib.sha256
        ).hexdigest()
        if not hmac.compare_digest(expected_mac, received_mac):
            return False, "Invalid ticket"
        data = json.loads(raw.decode())
```

```

    now = int(time.time())
    if now > data["expires_at"]:
        return False, "Ticket expired"
    if data["ticket_id"] in used_tickets:
        return False, "Replay detected"
    used_tickets.add(data["ticket_id"])
return True, "Authenticated: " + data["client_id"]
except Exception:
    return False, "Malformed ticket"
# KDC generates a service ticket
ticket = create_ticket("Client_A")
print("Generated Ticket:")
print(ticket)
# Server validates first use
print("\nFirst validation:")
print(validate_ticket(ticket))
# Replay attempt
print("\nReplay attempt:")
print(validate_ticket(ticket))

```

Expected Output

```

Generated Ticket:
[encoded ticket and MAC]
First validation:
(True, 'Authenticated: Client_A')
Replay attempt:
(False, 'Replay detected')

```

Replay Attack Analysis

- A captured ticket may be replayed by an attacker if the protocol does not check freshness.
- This simulation uses a unique ticket ID and a server-side used-ticket set to reject repeated use.
- The expiration time limits the lifetime of a ticket.
- The HMAC prevents an attacker from modifying the ticket contents without detection.
- Real Kerberos uses timestamps, authenticators and session keys; it is more sophisticated than this laboratory simulation.

Limitations

This program is a simplified simulation and does not implement the complete Kerberos AS/TGS protocol. It is intended to demonstrate ticket issuance, validation and replay protection concepts rather than to provide a deployable authentication system.

Result

The simulated server accepted a valid ticket once and rejected a second use of the same ticket as a replay, demonstrating the basic concept of replay prevention.

5. ElGamal Digital Signature Scheme

Aim

To implement an educational ElGamal digital signature scheme using SHA-256 for message hashing and demonstrate signing and verification.

Theory

ElGamal signatures are based on the difficulty of solving discrete logarithms in a suitable finite group. A private key is used to generate a signature and a public key is used for verification.

Python Program

```
import hashlib
import secrets
import math
# Small parameters for demonstration only
p = 467
g = 2
# Key generation
private_key = secrets.randbelow(p - 2) + 1
public_key = pow(g, private_key, p)
def hash_message(message):
    digest = hashlib.sha256(message.encode()).digest()
    return int.from_bytes(digest, "big")
def sign(message):
    while True:
        k = secrets.randbelow(p - 2) + 1
        if math.gcd(k, p - 1) != 1:
            continue
        r = pow(g, k, p)
        k_inv = pow(k, -1, p - 1)
        s = ((hash_message(message) -
            private_key * r) * k_inv) % (p - 1)
        if s != 0:
            return r, s
def verify(message, signature):
    r, s = signature
    if not (0 < r < p and 0 < s < p - 1):
        return False
    left = (pow(public_key, r, p) *
        pow(r, s, p)) % p
    right = pow(g, hash_message(message), p)
    return left == right
message = "ElGamal signature laboratory"
signature = sign(message)
print("Private key:", private_key)
```

```
print("Public key :", public_key)
print("Message  :", message)
print("Signature :", signature)
print("Valid    :", verify(message, signature))
tampered = "Modified message"
print("Tampered valid:", verify(tampered, signature))
```

Expected Output

```
Private key: [random value]
Public key : [derived value]
Message   : ElGamal signature laboratory
Signature : (r, s)
Valid     : True
Tampered valid: False
```

Security Evaluation

Feature	ElGamal Signature	Modern Standard Practice
Security basis	Discrete logarithm	Depends on selected standardized scheme
Message integrity	Yes	Yes
Authentication	Yes	Yes
Random nonce	Essential	Must be generated securely
Parameter size here	Tiny / educational	Large standardized parameters
Production use	Not this code	Use vetted libraries and standards

A particularly important property is nonce security. The signing value k must be unpredictable and must not be reused. Poor nonce handling can expose the private key. The small prime used in this example is deliberately insecure and should never be used for real signatures.

Result

The ElGamal signature successfully verified the original message and rejected the modified message, demonstrating digital-signature integrity and authentication.

Final Conclusion

The five Python experiments demonstrate complementary cryptographic mechanisms. Hash functions are useful for fingerprints and integrity, but MD5 is obsolete for collision-resistant security and SHA-256 is a much stronger choice. Digital signatures such as RSA and ElGamal provide public-key authentication and integrity, while HMAC provides efficient shared-secret authentication. Kerberos demonstrates how centralized ticket-based authentication can control access and reduce replay attacks through freshness and limited ticket lifetimes. For real-world applications, cryptography should be implemented using established standards, secure parameter sizes, proper key management, secure random number generation, and well-tested libraries rather than educational code.

Practical Security Note

The RSA and ElGamal parameters in this document are intentionally small so that the mathematics is easy to understand in a laboratory setting. The Kerberos implementation is a conceptual simulation, not a complete Kerberos protocol. Do not use these implementations or parameters for real authentication, digital signatures, password storage, or other production security purposes.